Small-GPT Code Guide

A module-by-module explanation of the current repository

This guide explains the current repository source code: what each module is for, how its main sections work, how data and tensors move through the system, and which limitations matter when interpreting results. It is written against the code present in this checkout. The program source is authoritative if this guide and an older comment or PDF disagree.

1. What the project does

Small-GPT is a compact, decoder-only Transformer project. Its primary workflow is:

1. Put permitted source files in an input directory.
2. Run `dataset.engine` to extract and clean text, record source metadata, make retrieval chunks, train or reuse the tokenizer, split sources, and write binary token streams.
3. Run `training.train` to train, resume, or update a model using those token streams.
4. Run `evaluation.py` to measure next-token prediction loss on held-out text, or `generation.generate` to sample text.
5. Optionally run `web.server` to serve a local chat UI with lexical retrieval over the source chunks.

The model predicts likely continuations. It does not guarantee truth, factuality, reasoning ability, or domain expertise. Retrieval can supply source text at request time but does not modify model weights.

2. Repository map

Path	Purpose and relationships
`README.md`	Setup, experiment workflow, CLI examples, and research limitations.
`requirements.txt`	Python package dependencies used across ingestion, model training, web serving, and optional synthetic-text generation.
`dataset/engine.py`	Main end-to-end data builder. Reads input files and optional URLs; writes `data/raw/*` and `data/processed/*`; uses `tokenizer/bpe.py`.
`dataset/collect.py`	Older, simpler collector for `data/text/*.txt`, `data/topics/**/*.txt`, and URLs. Writes source manifest, corpus, state, and new IDs. It is not what the README's main command invokes.
`dataset/chunk.py`	Older standalone chunker for the manifest. The main engine has its own better boundary-aware chunking implementation.
`dataset/prepare.py`	Older standalone tokenizer/binary preparation path. The main engine combines this work with extraction and provenance.
`dataset/README.md`	Short data-engine notes; some paths reflect the older modular workflow.
`tokenizer/bpe.py`	Byte-level BPE tokenizer: training, encoding, decoding, and persistence. Used by engine, trainer, and generation/web code.
`tokenizer/bpe.py.backup`	A commented-out historical copy. It is not imported by the application.
`model/modern_gpt.py`	RMSNorm, RoPE, grouped-query attention, SwiGLU, decoder blocks, model forward pass, loss, and sampling.

Path	Purpose and relationships
`model/count.py`	Instantiates selected model configurations and reports parameter counts. It is a utility, not part of training.
`training/config.py`	Default model dimensions and optimizer/training settings consumed by `training/train.py`.
`training/train.py`	Main training loop; reads binary data, constructs batches, runs optimizer steps, evaluates, and atomically saves checkpoints.
`evaluation.py`	Reproducible held-out loss/perplexity report with hashes and source split metadata.
`generation/generate.py`	Loads checkpoint/tokenizer and generates from a CLI prompt.
`generate_text.py`	Optional Ollama synthetic-content tool; saves generated text and a JSON sidecar. It does not feed the main pipeline automatically.
`pipeline/learn.py`	Convenience orchestrator for adding topic text and running an update/evaluation.
`rag.py`	In-memory lexical retrieval index over `data/raw/chunks.jsonl`, using a BM25-like score.
`web/server.py`	FastAPI backend; loads model once, exposes health/search/stream routes, and serves frontend files.
`web/index.html`, `web/style.css`, `web/app.js`	Browser interface: prompt form and controls, styling, and streamed response display.
`experiments/benchmark.py`	Prints dataset metadata and checkpoint validation metadata if present. It does not run a fresh evaluation.
`tests/test_data_engine.py`	Unit tests for selected normalization, filtering, chunking, local extraction, and source-ID helpers.
`data/ingest/`	Intended user-managed input tree for the main engine. The engine scans recursively for PDF, TXT, HTML, and HTM.
`data/raw/`	Source manifest, normalized corpus, retrieval chunks, state, stats, and new-source IDs.
`data/processed/`	Tokenizer vocabulary, train/validation/update/replay binaries, split record, and hashes/metadata.
`data/text/`	Text and sidecars currently in the checkout. The main engine does not ingest this directory by default; `dataset.collect` does.
`data/topics/`	Intended update input location in the README and `pipeline.learn`; the main engine scans it only when passed as `--input`.
`checkpoints/`	Model and optimizer checkpoint files written by training. A checkpoint must be paired with its tokenizer vocabulary.
`Documentation/`	Existing mathematical-intuition PDF and this code guide.

The empty `__init__.py` files mark Python packages so commands such as `python -m dataset.engine` and `python -m generation.generate` work. They contain no runtime logic.

3. Data products and IDs

The main engine emits these central files:

- `data/raw/sources.jsonl`: one JSON object per accepted source. Fields include stable `id`, display `label`, source `kind`, normalized text `sha256`, character count, and full extracted `text`.
- `data/raw/chunks.jsonl`: overlapping retrieval chunks, linked to source ID and carrying source label, kind, content hash, chunk index, and text.
- `data/raw/corpus.txt`: readable concatenation of source labels and source text for inspection.
- `data/raw/new_sources.json`: IDs newly introduced in this build relative to the existing manifest.
- `data/raw/state.json`: known source IDs and schema/version information.
- `data/raw/stats.json`: counts of files, accepted/skipped sources, duplicates, chunks, and related build statistics.

- `data/processed/vocab.json`: tokenizer's byte vocabulary and ordered merge rules. This is required to interpret token IDs.
- `data/processed/train.bin`, `val.bin`, `new.bin`, `replay.bin`: concatenated native-endian unsigned 16-bit token IDs as written by Python's `array('H')`. The files have no per-source separators; sampling windows may cross the concatenation boundary between sources. These files are intended for the same machine/runtime conventions used to build and read them; they are not a portable interchange format with an explicit byte-order header.
- `data/processed/split.json`: source IDs assigned to train and validation.
- `data/processed/meta.json`: token counts, hashes, vocabulary size, split information, and schema version.

Token IDs 0–3 are reserved special IDs; bytes use IDs 4–259; learned BPE symbols start at 260. Because binaries use 16-bit unsigned integers, every token ID must fit in that range. The configured 16,384-token vocabulary does.

4. The end-to-end data builder: `dataset/engine.py`

4.1 Imports, constants, and paths

The module imports CLI parsing, hashing, JSON, regular expressions, timestamps, Unicode normalization, filesystem paths, HTTP (`requests`), HTML parsing (`BeautifulSoup`), PDF extraction (`PdfReader`), and `BPETokenizer`. `ROOT` is derived from the source file location, so normal paths do not depend on the shell's current directory. Constants define input/output paths, user-agent, supported file extensions, default vocabulary size (16,384), chunk width (1,800 characters), and overlap (250 characters).

4.2 Stable normalization and quality filtering

`sha256(text)` hashes the UTF-8 encoding of normalized text. It gives the engine a content fingerprint for exact duplicate detection and change tracking; it is not a similarity detector.

`normalize_text(text)` proceeds through HTML entity unescaping, newline normalization, null-byte removal, Unicode NFKC normalization, joining a word split by a hyphenated line wrap, collapsing repeated horizontal whitespace, reducing excessive blank lines, dropping non-printable control characters, and trimming ends. Example: `train-\ning` becomes `training`. These rules improve consistency, but can also alter layout or join text that was intentionally hyphenated.

`quality(text, min_chars)` rejects text below the caller's character threshold or with fewer than 35% alphabetic characters. It is a deterministic extraction-quality heuristic, not a check of correctness, licensing, or usefulness.

4.3 Extraction

`extract_pdf(path)` uses pypdf to extract text page by page, normalizes each page, labels nonempty pages as `[Page N]`, and joins them with blank lines. Scanned image PDFs may yield little or no text because OCR is not performed.

`extract_html_text(text)` parses HTML and removes script/style/template-like or navigation elements (`script`, `style`, `noscript`, `svg`, `nav`, `footer`, `header`, `form`, `aside`) before getting visible text separated by newlines.

`read_local(path)` dispatches on extension. PDFs use the PDF extractor; TXT is read as UTF-8 with invalid bytes ignored; HTML/HTM is parsed. It returns `(text, kind)`.

`scrape(url, timeout=30)` downloads a URL with the project user-agent and timeout, errors on unsuccessful HTTP status, and checks content type or `.pdf` suffix. PDF bytes are passed through an in-memory byte stream to pypdf; other responses are parsed as HTML. Network collection can fail because of access controls, site rules, malformed pages, or connectivity.

4.4 Stable identities and old state

`canonical_file_id(path, root)` makes IDs relative to the input root, e.g. `file:papers/book.pdf`. This avoids embedding the full machine-specific input path in a normal ID.

`load_previous()` reconstructs an ID-to-row map from the existing JSONL manifest. Invalid JSON lines are skipped. `write_jsonl()` serializes iterable rows one JSON object per line with Unicode preserved.

During `main`, the existing manifest is loaded and its IDs captured as `previous_ids`. A content-hash-to-ID map supports cross-source exact deduplication. The nested `add(...)` function normalizes, filters, hashes, skips unchanged source versions, and skips duplicate content from a different source. If an existing source ID has changed content, a

new version ID ending in `#sha256:<12 hex digits>` preserves the previous version. Accepted newly introduced IDs go into `new_sources.json`.

4.5 Chunking for retrieval

`chunk_text(text, chars, overlap)` validates that overlap is smaller than chunk size, then yields trimmed slices. It prefers a paragraph boundary, otherwise a space, and carries an overlap into the next window. This keeps nearby chunks contextually connected. Character-based chunks are not guaranteed to match sentence or token boundaries.

`build_chunks(rows, chars, overlap)` writes one JSONL row for each chunk with source lineage, hash, and stable chunk index. The web retriever consumes this file; training consumes the full source text, not these retrieval chunks.

4.6 Tokenization, split, and binaries

`write_training_binaries(...)` either loads the fixed existing vocabulary (`--reuse-tokenizer`) or trains BPE on joined source text and saves the vocabulary. It warns when the corpus has fewer than 100,000 characters. It rejects a vocabulary larger than the requested limit.

Each source is encoded independently. Sources with fewer than 769 tokens are excluded from train/validation eligibility. The ID hash modulo 100 makes a deterministic approximate source-level split; source-level splitting reduces direct source leakage. If hashing puts all qualifying sources on the same side, a deterministic fallback assigns one qualifying source to validation. At least two qualifying sources are needed.

Important detail: validation membership is selected by source ID, not by random token windows. However, exact duplicated content across different materials can still leak unless deduplication catches it; source-level partitioning alone does not prove independence.

The function forms update data from training sources only: IDs in `new_sources.json` become `new.bin`, all other training sources become `replay.bin`. Validation sources are excluded. If there is no new source set, the fallback reuses some old training data as `new.bin`; therefore a built `new.bin` does not by itself prove that genuinely new content exists. If there are new IDs but none landed in train, the build errors rather than silently training on validation material.

The nested `write_bin` serializes each token list as `array('H')` and reports token totals. It writes all lists consecutively without source boundary markers. `file_sha` hashes outputs in 1 MiB blocks. The engine then writes `split.json`, followed by `meta.json` containing sizes and SHA-256 hashes.

4.7 CLI orchestration

`main()` defines input path, URL-list path, `--no-web`, `--reuse-tokenizer`, minimum characters, chunk width/overlap, vocabulary size, and validation percentage. It creates output directories, recursively enumerates supported local files in sorted order, and catches/logs per-file extraction failures. Unless `--no-web` is set, it visits nonblank, non-comment URLs and pauses briefly between requests. It writes manifest/state/corpus, builds retrieval chunks and binaries, validates the validation percentage, writes stats, and prints a summary.

The `if __name__ == '__main__'` guard runs this CLI only when invoked as a script/module, not when imported by tests.

5. Other data scripts

`dataset/collect.py`

This is an earlier collector. Its constants point to `data/text/*.txt`, `data/topics/**/*.txt`, and `data/links.txt`; unlike the current engine, it does not recursively extract PDFs or HTML from an arbitrary `--input` directory. It has its own `clean`, `quality`, `scrape`, `read_rows`, and `main` flow. Its IDs are based on resolved absolute file paths for text files, and it records a changed source under a content-version ID. The output overlaps the engine's `sources.jsonl`, `corpus.txt`, `new_sources.json`, and `state.json`; do not casually alternate the two collectors because their ID/schema behavior is not identical.

dataset/chunk.py

Reads each manifest row, makes fixed 1,800-character windows with 250-character overlap, and writes chunk JSONL. Unlike `engine.chunk_text`, it does not seek paragraph/space boundaries. It is a standalone predecessor; the end-to-end engine calls `build_chunks` itself.

dataset/prepare.py

Reads the manifest, trains or reuses BPE, encodes sources of at least 769 tokens, assigns source-level train/validation using `sha256(id) % 20`, writes binaries, and records a smaller metadata object. Its `--reuse-tokenizer` option supports a stable vocabulary for updates. The main engine contains a more complete version of these responsibilities, including hashes/split metadata and deduplication safeguards.

6. Byte-level BPE: `tokenizer/bpe.py`

6.1 Data model and byte reconstruction

`BPETokenizer` defines special IDs `PAD=0`, `UNK=1`, `BOS=2`, `EOS=3`, then 256 raw bytes at offset 4. Every learned merge stores a pair `(left_id, right_id)`; merge number `r` has ID `260+r`. `merge_rank` maps a pair to its order, and `_byte_cache` memoizes the byte string represented by a token.

`_symbol_bytes(token)` returns one original byte for a raw-byte token. For a learned token it recursively concatenates the bytes of its left and right component. Special IDs have no byte representation and invalid IDs error. `_rebuild_vocab()` creates a display vocabulary; it decodes bytes as Latin-1 only to preserve one-to-one byte values in JSON, not as the final user-facing UTF-8 text decode.

6.2 Training merges

`train(text, vocab_size, min_frequency, verbose)` validates arguments, UTF-8 encodes the corpus, and begins with one token per byte. Linked arrays `prev` and `nxt` represent the active token sequence without repeatedly shifting a Python list. Pair counts and sets of left positions are built by `_initial_pair_counts`; a max-priority heap is implemented with negative counts in `heapq`.

Each iteration discards stale heap entries until it finds the currently most frequent pair above `min_frequency`, assigns it the next merge ID, and applies the merge at every recorded position where the pair is still adjacent. Neighbor pair counts are decremented and updated around the merged location. The process stops at the requested vocabulary size, when no pair reaches minimum frequency, or when no merge applies. Large temporary structures are released and garbage collection is requested before the display vocabulary is rebuilt.

This is the core BPE idea: repeatedly replace the most frequent adjacent symbol pair with a new symbol. For example, byte symbols for repeated `t h` may become a symbol representing `th`, reducing the number of tokens needed for common sequences. The exact learned merges depend on the training corpus and are saved in order.

6.3 Encoding and decoding

`_encode_chunk(chunk)` converts UTF-8 bytes to byte-token IDs and repeatedly scans active neighboring pairs. It selects the available pair with the earliest learned merge rank, merges it through linked arrays, and repeats until no learned pair remains. This scan-based encoder uses less memory than maintaining a heap of stale candidates, at the cost of CPU time.

`encode(text, add_special_tokens=False, chunk_size=4400)` divides long input text into character slices, encodes each slice independently, and concatenates IDs. If requested, BOS and EOS are added. The normal data engine does not add these markers per source. Independent chunk encoding means a BPE pair cannot merge across a 4,400-character boundary.

`decode(ids)` recursively reconstructs bytes for ordinary tokens, skips PAD/UNK/BOS/EOS, and UTF-8 decodes with replacement for invalid sequences. `save(path)` writes vocabulary and ordered merges as compact JSON; `load(path)` restores them. The backup file is not used by imports.

7. Transformer implementation: `model/modern_gpt.py`

Use shapes `[B,T,C]` for batch, sequence, and model width. The configured model uses `B=2`, `T=768`, `C=768`, 14 layers, 12 query heads, 4 key/value heads, and a 2,048-wide feed-forward intermediate representation. `head_dim=C/n_heads=64`; each KV head is repeated for three query heads.

7.1 RMSNorm

`RMSNorm` learns one scale per feature. Given vector `x`, it computes `x / sqrt(mean(x²)+eps)`, then multiplies by learned weights. The square/mean is calculated in float32 for stability. Unlike LayerNorm, it does not subtract the mean. Shape remains unchanged.

7.2 RoPE helpers

`rotate_half` splits the final feature dimension into two halves and swaps them with a sign change. `apply_rope(q,k)` builds position-dependent sinusoidal angles from token positions and inverse frequencies, then rotates query and key vectors. Query/key shapes are `[B, heads, T, head_dim]`; the cosine/sine tables broadcast across batch and heads. Positional information therefore enters attention scores without a learned positional embedding table.

7.3 SwiGLU

`SwiGLU` has bias-free `gate`, `up`, and `down` linear layers. It computes `SiLU(gate(x)) * up(x)`, then projects from `ffn_dim` back to `dim` and applies dropout. The elementwise gate lets each feature modulate the other branch. For `[B,T,768]`, both expansion branches produce `[B,T,2048]`, and `down` returns `[B,T,768]`.

7.4 Grouped-query causal attention

`GQAAttention.__init__` checks that query-head count is divisible by KV-head count, derives head width and repeat factor, and creates Q/K/V/output projections. For configured dimensions:

- Q projection: `[B,T,768] -> [B,12,T,64]`
- K and V: `[B,T,768] -> [B,4,T,64]`
- RoPE rotates Q and K.
- `repeat_interleave(3, dim=1)` makes K/V have 12 heads, with groups of three query heads sharing one KV representation.
- `scaled_dot_product_attention(..., is_causal=True)` prevents position `t` from looking at positions after `t`. Dropout is enabled only during training.
- Output is transposed/reshaped to `[B,T,768]`, then passed through `o_proj`.

Conceptually each query head scores keys using scaled dot products, turns scores into probabilities, and takes a weighted sum of values. GQA reduces the K/V projection and cache dimensions compared with one separate K/V pair per query head. This implementation explicitly repeats K/V heads before attention, so the speed/memory benefit may be smaller than a kernel with native GQA support.

7.5 TransformerBlock

Each block is pre-normalized and has two residual updates:

```
x = x + Attention(RMSNorm(x))
x = x + SwiGLU(RMSNorm(x))
```

The residual paths preserve the incoming representation while each sublayer learns a correction. Input and output are `[B,T,C]`.

7.6 ModernGPT construction and forward pass

The constructor creates a token embedding table `[vocab_size, dim]`, `n_layers` blocks, final RMSNorm, and a bias-free vocabulary projection. The output projection weight is tied to the input embedding weight, so the same matrix maps token IDs to vectors and hidden states back to vocabulary scores. `_init_weights` initializes Linear and Embedding matrices from a normal distribution with standard deviation 0.02.

`forward(idx, targets=None)` expects integer IDs `[B,T]` and errors when `T > block_size`. It embeds IDs to `[B,T,C]`, runs blocks, applies final norm, and makes logits `[B,T,vocab_size]`. During training, if `gradient_checkpointing` is enabled, each block's activations are recomputed in backward to trade extra compute for lower activation memory.

When targets `[B,T]` are passed, logits are flattened to `[B*T,vocab_size]` and targets to `[B*T]`; cross entropy computes the mean negative log probability of the correct next token. Training code supplies shifted input/target windows, so logit position `t` is scored against token at original position `t+1`. Lower loss means better probability assigned to the held-out next tokens. Loss is measured in nats/token.

7.7 Autoregressive `generate`

The no-gradient method sets evaluation mode and repeats up to `max_new_tokens` times. It uses only the newest `block_size` IDs, obtains logits, selects the final position, applies an optional repetition penalty, divides by temperature, masks all but the top-k tokens if configured, optionally applies nucleus/top-p truncation, converts scores to probabilities, samples one token with `multinomial`, and appends it. Lower temperature concentrates probability; higher temperature spreads it. Sampling remains stochastic unless the distribution is effectively one-hot or RNG is fixed.

The repetition penalty adjusts logits for tokens present in the current context: negative logits are multiplied, positive logits divided by the penalty. The implementation does not stop on EOS; the CLI generation loop continues to the token limit, while the web server explicitly stops when it samples EOS.

8. Training settings and loop

`training/config.py`

`MODEL_SIZE='3060'` selects the 16,384 vocabulary, 768 context, 768 hidden width, 14 layers, 12 query/4 KV heads, FFN width 2,048, and dropout 0.05. Other constants set batch size 2, accumulation 16, base learning rate `3e-4`, update learning rate `6e-5`, AdamW betas `(0.9,0.95)`, weight decay 0.1, up to 10,000 steps, 500 warmup steps, evaluation every 250 steps over 20 batches, gradient clipping at 1.0, checkpointing on, minimum LR fraction 0.1, seed 1337, and replay sampling probability 0.35.

Effective tokens per optimizer step are `2 * 16 * 768 = 24,576`. This is an approximate effective batch size; the update mode chooses each microbatch from new or replay data independently, so realized replay proportion fluctuates.

`training/train.py`: helper functions

- `sha(path)`: SHA-256 hash of a file, used to bind checkpoints to the tokenizer.
- `load_bin(name)`: opens a `uint16` binary as a NumPy memory map, avoiding loading the entire corpus into RAM.
- `batch(data, block, bs, device, generator)`: samples random start offsets and builds `x` and one-token-shifted `y`. For `block=768`, each has shape `[2,768]`; `x[i,j]` predicts `y[i,j]`. It errors if the stream is too short.
- `amp(device)`: returns CUDA float16 autocast when running on CUDA, disabled on CPU.
- `evaluate(...)`: switches to eval mode, scores deterministic validation windows using a seeded generator, averages loss, and returns to train mode. It does not compute gradients.
- `atomic(payload,path)`: writes a temporary checkpoint in the target directory, flushes and syncs it, then atomically replaces the destination. This reduces the chance of leaving a partial checkpoint after interruption.
- `lr(step,total,peak)`: linearly warms up, then applies cosine decay toward `min_lr_ratio * peak`.
- `payload(...)`: stores model and optimizer states, step, best validation loss, model config, tokenizer hash, RNG states, and version.
- `restore_rng(state)`: restores Python, NumPy, Torch, and optional CUDA random number state on resume.

Main routine

The CLI accepts `--mode new|resume|update` and optional `--steps`. It seeds Torch and NumPy, selects CUDA if available, prints GPU information, enables TF32 on CUDA, checks data files, loads tokenizer/hash and validation stream, and copies the selected model config while replacing `vocab_size` with the actual tokenizer size.

- **new:** initializes model and AdamW from scratch, uses `train.bin`, base LR, step 0, and infinite best-loss sentinel.
- **resume:** loads `gpt.pt`, refuses a tokenizer-hash mismatch, restores model/optimizer/RNG and continues from checkpoint step + 1 using `train.bin`.
- **update:** loads `gpt.pt`, checks tokenizer hash, restores model weights but creates a fresh optimizer at update LR, and samples batches from `new.bin` or `replay.bin`. It resets update step/best tracking. It evaluates against the currently built `val.bin`.

The loop computes the learning rate, then for each accumulation microstep chooses a data stream, builds `x/y`, runs forward under autocast, divides loss by accumulation count, and calls backward. After the microsteps it unscales gradients, clips their norm, steps/updates the gradient scaler, and clears gradients. At each evaluation interval and final step it evaluates validation loss, writes `gpt-best.pt` if loss improved, and writes `gpt.pt`. Checkpoints include optimizer state for resume.

Practical code-reading cautions:

1. `steps = args.steps or config.max_steps` means passing `--steps 0` uses the default rather than doing zero steps.
2. `--mode update` overwrites the normal `gpt.pt`/`gpt-best.pt` paths. Preserve baseline copies before comparing conditions.
3. Update-mode evaluation uses the current validation binary. That is valid only if that split is meaningful for the question being studied.
4. Update mode loads model weights but does not restore the old optimizer state or RNG state; it is a fine-tuning run, not a bit-for-bit continuation.
5. CUDA AMP uses float16 and a GradScaler; CPU runs use full precision and may be very slow.
6. Resume restores optimizer state but the learning-rate helper receives a step offset that starts at zero for the resumed segment; the warmup/cosine schedule is therefore recalculated for the requested segment rather than continuing from the original global schedule position.

9. Evaluation, generation, and benchmark helpers

`evaluation.py`

Defines `sha256(path)` for large files in blocks. The CLI validates batch settings and required files, loads the checkpoint and vocabulary, checks the tokenizer hash, memory-maps `val.bin`, and uses the checkpoint's saved architecture. It samples fixed-length random windows from validation tokens with a reproducible seed, builds shifted `x/y` batches, evaluates under inference mode (and CUDA autocast on GPU), and averages per-batch cross entropy. Perplexity is `exp(mean_loss)`, the geometric-scale measure of next-token uncertainty. It prints and optionally writes JSON with checkpoint step, settings, loss, perplexity, validation/vocabulary hashes, dataset metadata, and source split.

This measures token prediction only. It does not measure factual response quality, retrieval support, or generalization beyond the chosen held-out corpus. Random windows are repeatable for a fixed seed/settings, though batches may overlap.

`generation/generate.py`

Parses prompt, maximum tokens, temperature, top-k, top-p, and repetition penalty. It selects CPU/GPU, loads the checkpoint and tokenizer, reconstructs the model from checkpoint config and state dict, encodes the prompt, keeps only the last context window, calls `ModernGPT.generate`, and decodes the complete prompt-plus-continuation. It assumes the default checkpoint and vocabulary paths exist.

`experiments/benchmark.py`

Loads and prints `meta.json` if present. If `gpt.pt` exists, it prints saved step/best loss and computes `exp(best_val)` (capped in the exponent at 20). It reports stored metadata rather than recomputing loss or validating the checkpoint.

10. Retrieval and the local web application

rag.py

`WORD` defines a simple English-centric token pattern, and `toks` lowercases its matches. `Retriever.__init__` calls `refresh`. `refresh` reloads JSONL chunks, builds per-chunk term frequencies (`_tf`), document frequencies, and an inverted posting list mapping each term to chunk indices.

`search(q, top_n)` scores each query term with an IDF factor and a BM25-like term-frequency/length normalization factor, then returns the highest-ranked chunks without the internal `_tf` field. It does not use semantic embeddings, stemming, multilingual tokenization, or a learned reranker. Repeated `refresh()` in each request makes changes to chunks visible, but rereads/reindexes the entire chunk file and may be expensive for a large collection.

web/server.py

At module import, it resolves paths, selects device, loads checkpoint/tokenizer/model, switches model to eval mode, initializes the retriever, creates FastAPI, and mounts static files. This means missing checkpoint/vocabulary prevents server startup.

`Req` validates request fields: prompt, retrieval toggle, token limit, temperature, top-k, and top-p. `/` returns the HTML app. `/health` reports status, device, parameter count, retrieval chunk count, and checkpoint step. `/search` refreshes the retrieval index and returns up to five matches.

`/stream` refreshes retrieval, optionally selects three chunks, builds a source-labeled context plus instruction and question, tokenizes/truncates to the model context, and returns a Server-Sent Events stream. Its generator repeatedly runs the model on the latest context window without gradients, applies temperature/top-k/top-p, samples and appends one token, decodes it, and yields JSON containing the token and source labels. It ends on EOS or the clamped token limit, then sends `[DONE]`. The browser receives source labels in each event, but `app.js` currently displays token text only.

Retrieval prompts can exceed the context window; truncation keeps the final encoded IDs, which may cut off the beginning of context or instruction. The model has no dedicated instruction-tuning guarantee, so the prompt wording is only a request to the pretrained/fine-tuned model.

Frontend files

- `index.html` creates page metadata, heading, chat area, prompt form, temperature and token inputs, retrieval checkbox, and imports CSS/JS.
- `style.css` sets the dark theme, centered layout, scrollable chat, message colors, textarea, and controls. It is intentionally compact and contains no application logic.
- `app.js` finds DOM elements, creates message nodes with `textContent` (so generated text is not interpreted as HTML), handles form submission, POSTs JSON to `/stream`, reads response chunks, splits SSE events on blank lines, parses each token event, appends token text, and scrolls the chat. It reports a generic server error for non-OK responses. It does not currently display retrieved source names or handle network exceptions with a user-facing message.

11. Synthetic text and convenience update orchestration

generate_text.py

`safe_filename` produces a portable lowercase underscore stem. `generate_text` constructs a structured educational-writing prompt and calls the Ollama Python client with the selected model and `num_predict`. `available_path` increments a suffix so existing text/sidecars are not replaced. `main` validates a positive output limit, creates the output directory, generates content, writes `.txt`, and writes `.json` metadata marking it synthetic and unreviewed with model/time/topic. Errors are printed to stderr and exit nonzero. These outputs are intentionally separate from the main engine's default `data/ingest` directory.

`pipeline/learn.py`

Parses required topic and optional steps, checks for a base checkpoint/tokenizer, converts topic to a filesystem slug, creates `data/topics/<slug>`, then invokes subprocesses from the repository root: engine with topics input/no web/fixed tokenizer, update training, and evaluation. It checks `new_sources.json` to avoid an update when nothing new was accepted. `run` prints the command and uses `check=True`, so subprocess failure stops the sequence. Current caveat: it evaluates `gpt-best.pt`, and update training may overwrite it; back up checkpoints before experimental comparisons.

12. Tests and dependencies

`tests/test_data_engine.py` uses `unittest` and temporary directories. It checks exact normalization, quality thresholds, multi-chunk output and maximum chunk width, TXT/HTML local extraction, and relative canonical IDs. It does not currently test BPE correctness, model math, the trainer, checkpoint resume, evaluation, retrieval ranking, or the web API.

`requirements.txt` lists PyTorch for tensor/model/training operations; NumPy for memory-mapped binary streams; requests and BeautifulSoup for web fetching/HTML parsing; FastAPI and Uvicorn for serving; pypdf for PDF text extraction; and Ollama for optional synthetic generation. CUDA support depends on the installed PyTorch build and host drivers; it is not a separate requirements entry.

13. One concrete training example

Suppose a token stream begins with IDs `[31, 52, 7, 19, ...]`, the sampled start is `i=100`, and the context length is 768. `batch` sets `x` to positions `100..867` and `y` to `101..868`. Both tensors have shape `[2,768]` for batch size 2. At every position, the model sees only tokens through that position because causal attention masks the future; it outputs 16,384 scores. Cross entropy compares these scores to the corresponding next IDs in `y`. Gradients flow backward through output projection, final norm, blocks, embeddings; gradient accumulation adds 16 microbatch gradients before one AdamW update.

For one attention layer, the hidden state `[2,768,768]` becomes Q `[2,12,768,64]` and K/V `[2,4,768,64]`; RoPE modifies Q/K based on positions; K/V repeat to 12 heads; causal attention returns `[2,12,768,64]`; heads combine back to `[2,768,768]`. The residual FFN expands to 2,048 channels and returns to 768. Fourteen such blocks produce final hidden states; the tied output matrix yields `[2,768,16384]` logits.

14. Execution flow at a glance

```
Local PDF/TXT/HTML or optional URL
        |
        v
dataset.engine: extract -> normalize -> filter -> hash/deduplicate -> manifest
        |                                                  |
        |                                                  +-> chunks.jsonl -> rag.Retriever -> web.server
        v
tokenizer.bpe: train/reuse BPE -> per-source token IDs -> source split -> binary files
        |
        v
training.train: sample x/y -> ModernGPT -> cross entropy -> backward/AdamW -> checkpoints
        |                                                          |
        +-> evaluation.py: held-out loss/perplexity                +-> generation.generate
```

15. Known limitations and code-level concerns

- The installed data tree may include text files under `data/text`, but the main engine defaults to `data/ingest`; directory presence does not imply ingestion.
- `dataset.collect`, `dataset.chunk`, and `dataset.prepare` overlap with the newer engine and have different defaults/IDs. Prefer the documented engine path for a consistent run.
- Binary files concatenate sources without separators. A sampled training window can straddle the end of one source and the start of another, effectively teaching a transition that did not occur in source text.
- `new.bin`/`replay.bin` have fallback behavior when one category is empty. Inspect `meta.json`, `new_sources.json`, and `split.json` before update runs.

- The source split is deterministic, but a small source count gives an approximate split that may differ substantially from the requested percentage. Similar content in separate sources can still cause leakage.
- The tokenizer encodes character chunks independently, adding boundaries where merges cannot cross. Encoding prioritizes bounded memory at a CPU cost.
- Attention repeats K/V heads in memory before calling generic scaled dot-product attention. This is simple and correct for the shared-head layout but may not realize all native GQA performance savings.
- `evaluation.py` samples windows with replacement, so reports are repeatable but not necessarily exhaustive over validation text.
- The model uses no KV cache. Generation reruns attention over the context for every new token, so longer output is computationally expensive.
- Web retrieval is lexical, the prompt may be truncated from the left, and the browser omits the source list even though the API returns it.
- The checked-in README notes the intended RTX 3060 profile, but actual speed/memory use depends on hardware, driver, PyTorch build, and corpus size.

16. Suggested reading order

1. Read this guide's data products section, then `dataset/engine.py` from extraction to `main`.
2. Follow `tokenizer/bpe.py` from token-ID layout to `encode`/`decode`.
3. Read `training/config.py`, then trace `batch`, `evaluate`, and the mode branches in `training/train.py`.
4. Trace a `[B,T]` tensor through `ModernGPT.forward`, then inspect attention and generation.
5. Compare `evaluation.py` with `generation/generate.py` to see the distinction between scoring and sampling.
6. Finish with `rag.py` and `web/server.py` for source-assisted inference.